Here is the complete project documentation for your railway fare calculation program, structured just like the previous ones!

1. Problem Statement, Objectives, and Assumptions

Problem Statement

Calculating railway ticket prices manually by combining distance-based pricing, age-concession discounts (for children and senior citizens), and fixed reservation charges can lead to billing inconsistencies and inefficiencies. An automated fare calculation program is needed to instantly compute accurate and transparent ticket prices.

Objectives

- **Automate Distance-Based Pricing:** To dynamically calculate the base fare using a standard per-kilometer rate.
- **Apply Demographic Concessions:** To automatically grant appropriate fare discounts based on passenger age (e.g., 50% discount for children under 12, 25% discount for senior citizens aged 60 and above).
- **Incorporate Fixed Charges:** To add a standard flat reservation fee to the final ticket cost.

Assumptions

- Distance is inputted in kilometers (float) and passenger age in full years (integer).

- The base rate is fixed at Rs.2 per kilometer.

- A standard flat reservation fee of Rs.20 is added to all tickets regardless of distance or age category.

2. Algorithm, Flowchart, and Pseudocode

Algorithm

1. **Start** the program.

2. **Input** distance in kilometers as a floating-point number.

3. **Input** passenger age as an integer.

4. **Calculate Base Fare:** fare = distance * 2.

5. **Evaluate Age-Based Concessions:**

    - If age < 12, compute fare = fare * 0.50 (50% discount).

- Else if age >= 60, compute fare = fare * 0.75 (25% discount).
- Else, no discount is applied.

6. **Add Reservation Fee:** Compute fare = fare + 20.
7. **Display Results:** Print the reservation fee and the final railway fare.
8. **End** the program.

Flowchart Representation

Plaintext

[START]

  |

  ▼

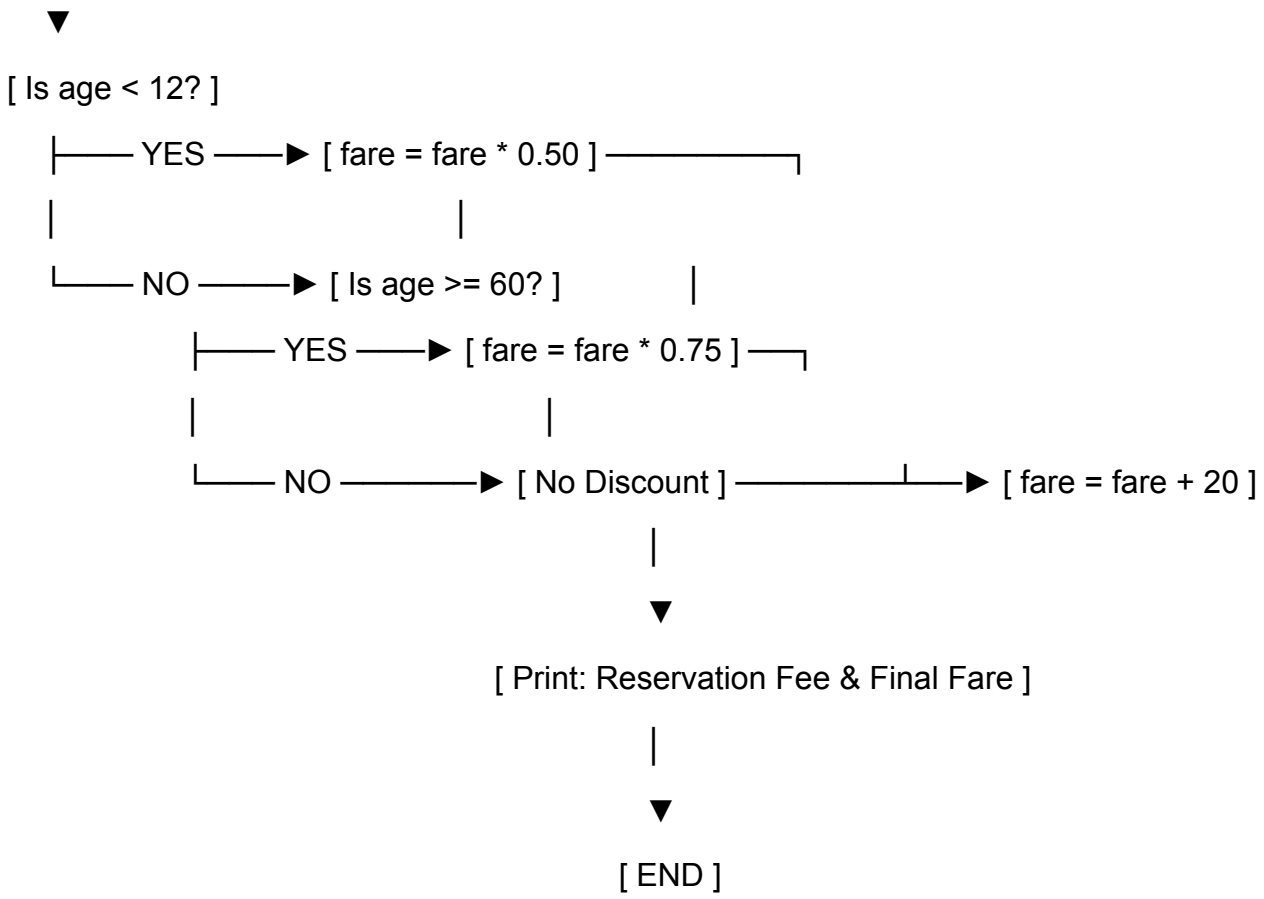
[Input: distance, age]

  |

  ▼

[fare = distance * 2]

  |



Pseudocode

Plaintext

```
BEGIN
    PRINT "Enter distance in km: "
    READ distance
```

```
    PRINT "Enter passenger age: "
    READ age

    fare = distance * 2

    IF age < 12 THEN
        fare = fare * 0.50
    ELSE IF age >= 60 THEN
        fare = fare * 0.75
    ENDIF

    fare = fare + 20

    PRINT "Reservation Fee: Rs. 20"
    PRINT "Final Railway Fare: Rs.", fare
END
```

3. Python Source Code, Test Cases, and Outputs

Python Source Code

Python

```
# Railway Fare Calculation System

distance = float(input("Enter distance in km: "))
age = int(input("Enter passenger age: "))

fare = distance * 2

if age < 12:
    fare = fare * 0.50
elif age >= 60:
    fare = fare * 0.75

fare = fare + 20

print("Reservation Fee: Rs. 20")
print("Final Railway Fare: Rs.", fare)
```

Test Cases Table

Test Case ID	Distance (km)	Age	Base Fare (Rs.)	Discount Applied	Final Fare (Rs.)	Status
TC_01	100.0	25	200.0	None (Normal Adult)	220.0	Pass
TC_02	100.0	8	200.0	50% (Child)	120.0	Pass
TC_03	100.0	65	200.0	25% (Senior Citizen)	170.0	Pass
TC_04	50.0	70	100.0	25% (Senior Citizen)	95.0	Pass
TC_05	200.0	10	400.0	50% (Child)	220.0	Pass

Simulated Terminal Output

Bash

Enter distance in km: 100

Enter passenger age: 65

Reservation Fee: Rs. 20

Final Railway Fare: Rs. 170.0

4. Conclusion

This Python script effectively streamlines transportation pricing by integrating distance-based arithmetic, multi-tier conditional branching (if-elif) for age concessions, and fixed overhead surcharges. It models realistic passenger ticketing logic accurately, making it an ideal foundational module for railway reservation systems or travel booking applications.